

INTRODUÇÃO

O TPC-E (Sigla para Transaction Processing Performance Council Benchmark E) faz a simulação de um ambiente de corretora de valores, cujo qual realiza inúmeras transações financeiras, como, por exemplo, compras e vendas de ações, gerenciamento de contas de clientes, atualização de dados de mercado, etc.[1]. O TPC-E é uma evolução do TPC-C (sigla para Transaction Processing Performance Council Benchmark C) um benchmark criado em 1992 para simular um ambiente de processamento de transações online em uma distribuidora atacadista, composto por cinco tipos de transações [2]. Apesar de antigo e com dependência de I/O em disco, ainda é muito utilizado por sua simplicidade e ampla disponibilidade em ferramentas como HammerDB [5]. Em 2007, o TPC-E foi lançado como sucessor natural, trazendo um modelo mais moderno baseado em uma corretora de ações, com dados mais realistas, menor dependência de armazenamento físico e maior complexidade para refletir melhor as infraestruturas atuais. Embora o TPC-E seja o mais indicado para cenários modernos, o TPC-C ainda é válido e continua relevante em ambientes acadêmicos e em testes rápidos [3].

Testes de benchmark precisam ter bastante rigor quanto ao padrão que se deve seguir, permitindo a inferência de dados e possíveis comparações entre sistemas quaisquer que sejam [7]. A documentação do TPC-E é bem detalhada e aborda de forma rígida como se deve proceder para realizar o benchmark dentro do que o conselho planejou como padrão. Não obstante, o conselho ainda exige que a arquitetura planejada para o benchmark seja enviada para avaliação pelos membros. Só então os testes realizados poderão compor a base de informações que se encontram no site do conselho.

Ainda na documentação do TPC-E, podemos encontrar na seção de definição as 12 transações que compõe o TPC-E, sendo elas:

- 1) **Broker-Volume** que foca em relatórios internos de desempenho de corretores, Seu Database Footprint mostra que as operações realizadas são *Return* e *Reference* para colunas nas tabelas BROKER e TRADE_REQUEST;
- 2) **Customer-Position** resgata o perfil do cliente e resume o valor dos ativos. O Database Footprint da transação mostra que as operações realizadas são exclusivamente *Return* e *Reference*;
- 3) **Market-Feed** Processa dados de mercado em tempo real e aciona ordens pendentes. Seu DB Footprint mostra que ela modifica colunas na tabela LAST_TRADE (colunas LT_DTS, LT_PRICE, LT_VOL) e TRADE (colunas T_DTS, T_ST_ID). Adiciona rows na tabela TRADE_HISTORY e remove da tabela TRADE_REQUEST;
- 4) **Market-Watch** monitora o desempenho geral do mercado e tendências de segurança. O seu DB Footprint lista os métodos de acesso como *Reference* para todas as tabelas acessadas (COMPANY, DAILY_MARKET, HOLDING_SUMMARY, INDUSTRY, LAST_TRADE, SECURITY, WATCH_ITEM, WATCH_LIST);
- 5) **Security-Detail** acessa informações detalhadas sobre uma segurança específica e seu DB Footprint lista os métodos de acesso como *Return* ou *Reference* para todas as tabelas e colunas acessadas;
- 6) **Trade-Lookup** recupera informações sobre um conjunto de negociações e seu DB Footprint mostra que as operações realizadas são exclusivamente *Return* ou *Reference*;

- 7) **Trade-Order** permite a compra ou venda de títulos com estimativa de impacto financeiro. Nessa transação, o DB Footprint mostra que ela adiciona uma row nas tabelas TRADE, TRADE_HISTORY e TRADE_REQUEST (a depender do caso);
- 8) **Trade-Result** completa uma negociação no mercado de ações e atualiza as carteiras dos clientes. O Database Footprint dessa transação indica que ela modifica colunas em tabelas como BROKER e TRADE, e adiciona linhas em CASH_TRANSACTION, HOLDING_HISTORY, SETTLEMENT e TRADE_HISTORY. Além disso, pode (a depender do caso) remover linhas das tabelas HOLDING e HOLDING_SUMMARY;
- 9) **Trade-Status** fornece o status de um conjunto específico de negociações. O DB Footprint lista todas as operações como Return;
- 10) **Trade-Update** realiza pequenas correções ou atualizações em negociações históricas. O DB Footprint indica que ela modifica colunas nas tabelas CASH_TRANSACTION (coluna CT_NAME), SETTLEMENT (coluna SE_CASH_TYPE) e TRADE na coluna T_EXEC_NAME;
- 11) **Data-Maintenance** simula modificações periódicas em dados estáticos de referência. O DB Footprint lista explicitamente a operação "*Modify*" para diversas tabelas, como ACCOUNT_PERMISSION, ADDRESS, COMPANY, CUSTOMER, CUSTOMER_TAXRATE, DAILY_MARKET, EXCHANGE, FINANCIAL, NEWS_ITEM, SECURITY, TAXRATE e WATCH_ITEM; Finalmente,
- 12) **Trade-Cleanup** cancela negociações pendentes ou submetidas, usado antes do Test Run. o DB Footprint indica que ela modifica colunas na tabela TRADE (como T_DTS e T_ST_ID), adiciona linhas na tabela TRADE_HISTORY e remove linhas da tabela TRADE_REQUEST.

Como visto, são doze transações com tarefas claras, inclusive com “subtarefas” para que a transação seja corretamente finalizada, chamadas de “Frames”. A ideia dos frames é dividir a transação em etapas como acontece realmente em uma aplicação. Isto porque, uma aplicação real, primeiro lê um valor no banco de dados (SELECT), após essa leitura, a aplicação altera os dados lidos e, conseqüentemente, salvará essas alterações (UPDATE). Também pode haver inserções como cadastros, por exemplo, e exclusões que, nesse caso, é comum a aplicação lê a informação para depois excluir. O fato é que o projeto do TPC-E, tenta atender a muitos casos que são comuns nas aplicações se comunicando com bancos de dados.

Neste trabalho, foi utilizado o TPC-E para avaliação de desempenho em 2 SGBD largamente utilizados no mercado, SQL Server e PostgreSQL, devido a qualidade e quantidade de recursos ofertados aos administradores de bancos de dados. O SQL Server é de domínio da Microsoft, enquanto o PostgreSQL é de código 100% aberto mantido por uma comunidade de desenvolvedores coordenados pelo Grupo de Desenvolvimento Global do PostgreSQL [4]. O fato de ser de código aberto desde os primórdios de sua existência, faz com que o PostgreSQL seja constantemente estudado, avaliado e adaptado para uso específicos como, por exemplo, a Amazon com o projeto Aurora utiliza uma base do código do PostgreSQL, o que faz com que o serviço seja facilmente compatível com o SGBD [6].

EXPERIMENTOS

Nesta seção, serão descritos os passos iniciais para a execução dos benchmarks, bem como os achados técnicos durante o processo.

Planejamentos Iniciais dos Experimentos

A ideia inicial era construir um código com doze códigos distintos, um para cada transação e um principal que acionaria as transações de acordo com um valor de duração. Cada código teria uma função para cada “frame” da transação. Cada função receberia como parâmetros um cursor instanciado a partir de uma conexão com o SGBD e um vetor de informações caso seja necessário como, por exemplo, as funções de update precisariam de valores para realizar os UPDATES.

Contudo, ter os códigos com funções corretas e obedecendo às diretrizes do documento do TPC-E, não é suficiente para realizar o benchmark. Também é necessário um “populated database”, contudo, não consideramos clara a documentação em ensinar como utilizar a ferramenta EGenLoader para construir nossa própria base de dados populada. Uma ideia criativa e mais rápida foi usar um backup do banco de dados utilizado na ferramenta Benchmark Factory, pois a mesma gera o banco de dados e carrega dados dentro dele. Como na ferramenta há a possibilidade de gerar os dados sem rodar os experimentos, o banco gerado pelo Benchmark Factory não foi alterado e, portanto, encontra-se no estado como a documentação do TPC-E rege.

Rodamos dois containers com imagens do PostgreSQL e do SQL Server e restauramos os backups dos bancos gerados no Benchmark Factory. Executamos os containers expondo as portas correspondentes aos bancos e realizamos a conexão externa com o DBeaver para validar a qualidade da conexão e integridade dos dados. Estando tudo funcionando como esperado, passamos à conexão dos códigos python para as transações. Um exemplo de código para a transação Market-Feed para o SQL Server pode ser vista a baixo:

```
import pyodbc
import random
import time
import datetime

def frame1(cur):
    updates = []
    cur.execute("SELECT TOP 100 S_SYMB FROM TPC_E.dbo.E_SECURITY
ORDER BY NEWID() ")
    symbols = [row[0] for row in cur.fetchall()]
    for sym in symbols:
        datetime_now = datetime.datetime.now()
        price = round(random.uniform(10, 500), 2)
        volume = random.randint(100, 10000)
        change = round(random.uniform(-5, 5), 2)
        bid = round(price - random.uniform(0, 0.5), 2)
```

```

        ask = round(price + random.uniform(0, 0.5), 2)
        updates.append((sym, datetime_now, price, volume, change,
bid, ask))

    return updates

def frame2(updates, cur):
    for sym, dt, price, vol, chg, bid, ask in updates:
        cur.execute("UPDATE TPC_E.dbo.E_LAST_TRADE SET LT_DTS = ?,
LT_PRICE = ?, LT_VOL = ? WHERE LT_S_SYMB = ?",
                    dt, price, vol, sym)

def frame3(updates, cur):
    for sym, dt, price, vol, chg, bid, ask in updates:
        cur.execute("UPDATE TPC_E.dbo.E_DAILY_MARKET SET DM_DATE =
?, DM_CLOSE = ?, DM_HIGH = ?, DM_LOW = ?, DM_VOL = ? WHERE DM_S_SYMB =
?",
                    dt.date(), price, price, price, vol, sym)

def frame4(updates, cur):
    for sym, dt, price, vol, chg, bid, ask in updates:
        cur.execute("""
            UPDATE TPC_E.dbo.E_SECURITY
            SET S_52WK_HIGH = CASE WHEN S_52WK_HIGH < ? THEN ?
ELSE S_52WK_HIGH END,
            S_52WK_LOW = CASE WHEN S_52WK_LOW > ? THEN ? ELSE
S_52WK_LOW END
            WHERE S_SYMB = ?""", price, price, price, price, sym)

def frame5(updates, cur):
    print()
    for sym, dt, price, vol, chg, bid, ask in updates:
        cur.execute("""
            INSERT INTO TPC_E.dbo.E_TRADE (
                T_DTS, T_ST_ID, T_TT_ID, T_IS_CASH, T_S_SYMB,
                T_QTY, T_CA_ID, T_EXEC_NAME, T_BID_PRICE, T_CHRG,
T_COMM, T_TAX, T_LIFO
            )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                    dt, 'SBMT', 'TMB', '1', sym,
                    vol, 1, 'MFEED', price, 0, 0, 0, '1') # Valores fixos
de exemplo

```

```

def entrada():
    conn = pyodbc.connect('DRIVER={ODBC Driver 17 for SQL
Server};SERVER=localhost;DATABASE=TPC_E;UID=sa;PWD=Abc.123*')
    cur = conn.cursor()
    updates = frame1(cur)
    frame2(updates, cur)
    frame3(updates, cur)
    frame4(updates, cur)
    frame5(updates, cur)
    conn.commit()
    cur.close()

# Testando a função de entrada
start = time.time()
entrada()
end = time.time()
print(f"Execution time: {end - start:.2f} seconds")

```

Na execução deste código, detectamos um primeiro problema que cuja a solução poderia violar a integridade do banco de dados do experimento. Perceba que na função “frame3”, há uma tentativa de atualização de um valor no banco com a seguinte query:

```

cur.execute("UPDATE TPC_E.dbo.E_DAILY_MARKET SET DM_DATE = ?,
DM_CLOSE = ?, DM_HIGH = ?, DM_LOW = ?, DM_VOL = ? WHERE DM_S_SYMB = ?",
            dt.date(), price, price, price, vol, sym)

```

Essa função retornava um erro de chaves duplicadas, o que nos levou a analisar a tabela E_DAILY_MARKET para investigar o que poderia estar acontecendo, mas, de antemão, imaginávamos que os dados contidos no banco, se valiam de inúmeras datas, por isso dava esse erro. Contudo, ao analisar a referida tabela, havia um índice composto no banco com duas chaves de busca, uma para DM_S_SYMB, que se trata de uma espécie de sigla, exemplo, “EHA”, e outra para DM_DATE, que se trata da data. Ao tentar fazer o mesmo update da função através de uma query, o mesmo erro era retornado, mesmo que se usasse uma data absurdamente improvável:

```

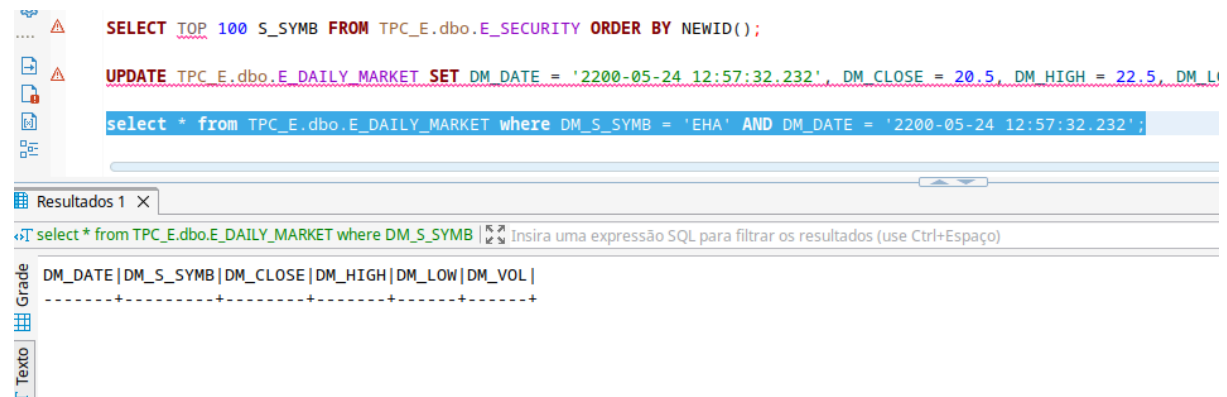
Erro SQL [2601] [23000]: Cannot insert duplicate key row in
object 'dbo.E_DAILY_MARKET' with unique index 'PK_E_DAILY_MARKET'. The
duplicate key value is (EHA , May 24 2200 12:57PM).

Posição do erro:

```

Para concluirmos que a data não realmente existia, ou que o par de chaves não era usado, fizemos a seguinte consulta:

```
select * from TPC_E.dbo.E_DAILY_MARKET where DM_S_SYMB = 'EHA' AND
DM_DATE = '2200-05-24 12:57:32.232';
```



Esse erro ao tentar executar o UPDATE na tabela E_DAILY_MARKET pode estar ligado à existência de um índice único (UNIQUE) definido sobre as colunas DM_S_SYMB e DM_DATE. Esse tipo de índice impõe a restrição de que não podem existir duas linhas com os mesmos valores nessas colunas. Decidiu-se experimentar a remoção do índice e criar um novo sem o UNIQUE, da seguinte forma:

```
DROP INDEX PK_E_DAILY_MARKET ON TPC_E.dbo.E_DAILY_MARKET;
CREATE CLUSTERED INDEX PK_E_DAILY_MARKET ON TPC_E.dbo.E_DAILY_MARKET (
DM_S_SYMB ASC , DM_DATE ASC )
WITH ( PAD_INDEX = OFF ,FILLFACTOR = 100 ,SORT_IN_TEMPDB = OFF ,
IGNORE_DUP_KEY = OFF , STATISTICS_NORECOMPUTE = OFF , ONLINE = OFF ,
ALLOW_ROW_LOCKS = ON , ALLOW_PAGE_LOCKS = ON )
ON [PRIMARY ] ;
```

Novamente, o SELECT retornou vazio para os valores de DM_S_SYMB, DM_DATE especificados. Então, ao tentar fazer o UPDATE, o mesmo erro foi retornado, contudo, em outro índice: E_DAILY_MARKET_INDX1. Nesse outro, índice as chaves de busca estão invertidas em relação ao primeiro e ele também é UNIQUE.

```
-- O cara criou dois indices, um clustered e um nonclustered, mas com
as chaves invertidas.

CREATE UNIQUE CLUSTERED INDEX PK_E_DAILY_MARKET ON
TPC_E.dbo.E_DAILY_MARKET ( DM_S_SYMB ASC , DM_DATE ASC )
WITH ( PAD_INDEX = OFF ,FILLFACTOR = 100 ,SORT_IN_TEMPDB = OFF ,
IGNORE_DUP_KEY = OFF , STATISTICS_NORECOMPUTE = OFF , ONLINE = OFF ,
ALLOW_ROW_LOCKS = ON , ALLOW_PAGE_LOCKS = ON )
ON [PRIMARY ]

CREATE UNIQUE NONCLUSTERED INDEX E_DAILY_MARKET_INDX1 ON
TPC_E.dbo.E_DAILY_MARKET ( DM_DATE ASC , DM_S_SYMB ASC )
```

```
WITH ( PAD_INDEX = OFF , FILLFACTOR = 100 , SORT_IN_TEMPDB = OFF ,  
IGNORE_DUP_KEY = OFF , STATISTICS_NORECOMPUTE = OFF , ONLINE = OFF ,  
ALLOW_ROW_LOCKS = ON , ALLOW_PAGE_LOCKS = ON )  
ON [PRIMARY ]
```

Esses detalhes nos levou a imaginar quantos casos como esse poderiam estar presentes no banco de dados, o que nos levaria a ter que sair alterando detalhes no banco que poderiam, em primeiro lugar, invalidar o banco para o Benchmark, segundo a comprometer a integridade do banco de dados. Dessa forma, decidimos usar a ferramenta da Quest, Benchmark Factory para todo o processo, desde a criação do banco até a execução dos experimentos.

Projeto com o Benchmark Factory

Passamos então a propor os experimentos utilizando a ferramenta da Quest, que oferece um tempo limitado de uso (Trial), mas consegue executar todo o processo do TPC-E. Após analisar a ferramenta e as possibilidades que a mesma oferece, optamos por organizar o benchmark em 3 etapas: Uma rodada com apenas um único worker, uma segunda com dois workers e a última com três workers de 2 minutos cada rodada. Durante todo o processo utilizando o Benchmark Factory, mantivemos o fator de impacto em 1, o padrão da ferramenta de benchmark. As etapas são comuns aos dois bancos de dados, SQL Server e PostgreSQL.

A execução dos experimentos na ferramenta pode ser programada, de forma que podemos colocar as 3 etapas já no pipeline de execução das tarefas. A ferramenta inicia criando o banco de dados com suas tabelas e estruturas, na sequência ela popula o banco de dados com os valores conforme as regras do TPC-E. Após o banco estar criado e populado, a ferramenta cria estruturas de índices e, por fim, executa os experimentos conforme a configuração que escolhemos.

Ambiente de Execução

A máquina utilizada para os experimentos tinha um Intel Core i7-10510U com 16GB de memória RAM rodando o Sistema Operacional Windows 11 atualizado na data 01 de agosto de 2025. Como o Sistema Operacional era Windows, o sistema de arquivos utilizado foi o NTFS desenvolvido pela Microsoft com base em projetos da IBM [8]. A versão utilizada para os experimentos do Benchmark Factory foi a 9.0.0. Todo o processo foi realizado pela referida ferramenta, sem nenhuma intervenção no banco ou na máquina durante o processo.

Execução dos Experimentos com o PostgreSQL

A execução foi iniciada às 14:06:01 de 01 de agosto de 2025 e teve um tempo decorrido de 06 minutos e 36 segundos. A sequência mencionada anteriormente foi seguida pela ferramenta: dois minutos para 1 worker, 2 minutos para 2 workers e 2 minutos para 4 workers. O tempo a mais da duração da execução se dá em decorrência da transição entre

uma etapa e outra que pode ser percebida nos gráficos. O detalhamento da versão do PostgreSQL utilizada é “PostgreSQL 17.5 x86_64-windows.

O throughput máximo do PostgreSQL foi encontrado na carga de 4 workers resultando em um número de transações por segundo (TPS) de 374,626. Nesse ponto, o tempo médio de resposta do servidor era de 0,007 segundos. Como o TPS máximo foi encontrado na última carga, ou seja, 4 workers, é possível que uma taxa de transferência maior pudesse ser alcançada com uma carga de usuários maior. Dessa forma, o conjunto de hardware mais o SGBD poderiam ter sido mais estressados para se extrair o máximo de desempenho do conjunto.

Worker(s)	TPS	Tempo de Resposta (Avg. s)	Tempo de Transação (Avg. s)	Total de Execuções	Total Rows	Total Erros
1	122,972	0,005	0,008	14733	997989	0
2	225,355	0,006	0,008	26849	1838030	1
4	374,626	0,007	0,010	44683	3062641	0

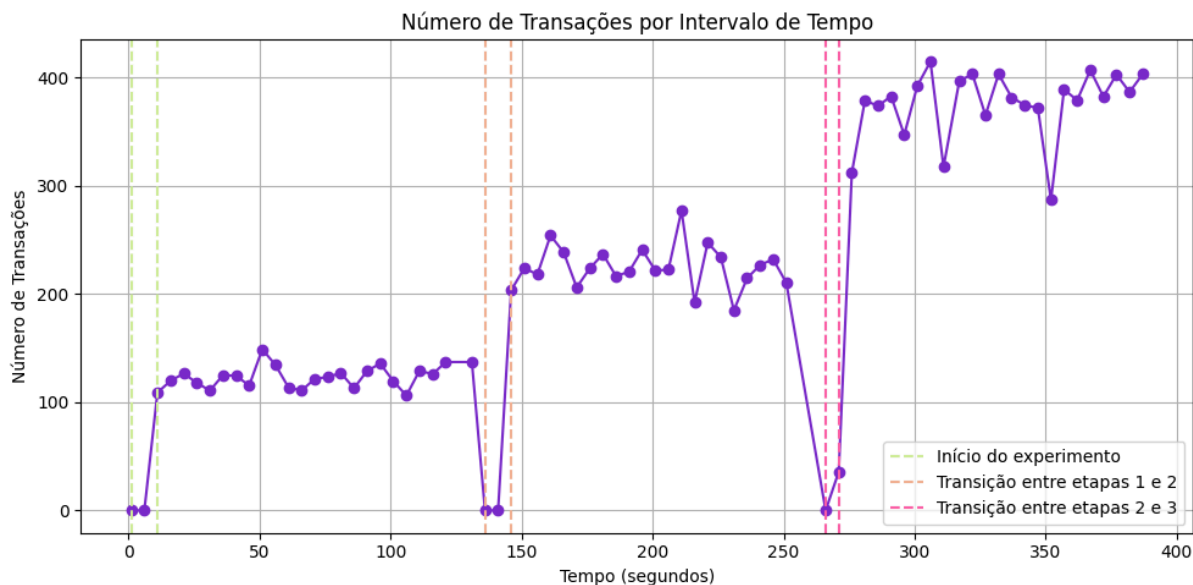
Na tabela acima, podemos observar a evolução do número de workers e, da mesma forma, a evolução do número de transações por segundo. Entre um e dois workers não há mudança se observado pelo tempo médio de transações contabilizado em segundos. Na última coluna e na linha da etapa de execução com 2 workers é possível perceber que há um erro detectado durante a execução. Ao analisar o log gerado pela própria ferramenta, que é bastante simplificado, percebemos que o erro é exatamente o mesmo erro apontando na subseção anterior. Ou seja, a tentativa de UPDATE na tabela E_DAILY_MARKET com um retorno exatamente igual ao erro que enfrentamos.

Na tabela abaixo, são apresentados os valores para total de transações contabilizadas por tipo de transação. Por exemplo, os tipos de transações Trade-Status e Market-Feed são as que mais impactam no número total de transações do experimento. Nessa mesma tabela, também podemos perceber que a ferramenta deixa de atender a duas das 12 transações do TPC-E: Data-Maintenance e Trade-Cleanup. No caso dessa última, a dispensa pode fazer sentido pois a cada nova execução do benchmark um novo banco é criado ou recuperado a partir de um backup que a própria ferramenta cria. Contudo, o tipo de transação “Data-Maintenance”, que a ferramenta deixa de utilizar, não aparenta fazer muito sentido. Isto porque ela é responsável por modificar de tempos em tempos dados que são usados principalmente para referência por outras transações.

Transaction Mix		
Transaction Name	Transaction Count	Mix %
Trade-Update	885	1,981
Trade-Status	8410	18,821

Transaction Mix		
Trade-Result	4410	9,870
Trade-Order	4866	10,890
Trade-Lookup	3549	7,943
Security-Detail	6199	13,873
Market-Watch	7990	17,882
Market-Feed	443	0,991
Customer-Position	5754	12,877
Broker-Volume	2177	4,872

O gráfico a baixo representa o número de transações por intervalo de tempo. Primeiramente, é importante destacar que os intervalos de queda nesse gráfico, marcados com uma linha vertical pontilhada, indicam a transição entre as etapas. Como as etapas foram configuradas na própria ferramenta, ela mesma atualiza as configurações para aumentar o número de workers a cada etapa. Contudo, a transição tem um certo custo de tempo aparentemente, talvez pelo modo como a ferramenta foi construída. Mas, desconsiderando essas faixas de transição e início, podemos perceber claramente a evolução do número de transações no decorrer do tempo a cada etapa. Como era de se esperar, quanto maior for o número de workers, maior é o número de transações. A linha do gráfico, na parte da primeira etapa, tem uma estabilidade perceptível, justificada pelo fato de haver apenas um worker realizando transações. Ao avançar para a segunda etapa com 2 workers, já começamos a perceber uma menor estabilidade que piora com 4 workers. Mostrando que o aumento do número de workers, mesmo que ainda pequeno, começa a causar alguma sobrecarga nos recursos, seja computacional, seja de banco de dados propriamente dito.



Execução dos Experimentos com o SQL Server

A execução foi iniciada às 15:20:37 de 01 de agosto de 2025 e teve um tempo decorrido de 06 minutos e 35 segundos. O throughput máximo do SQL Server foi encontrado na carga com 4 workers, como era de se esperar pelo que foi observado nos resultados do PostgreSQL, contudo o número de transações por segundo agora subiu para 588,212 com um tempo médio de resposta de 0,001 segundos. Esse tempo já pode ser considerado ótimo, o que poderá tornar as implementações de ajuste de performance muito mais sensíveis. A depender da forma como a ferramenta contabiliza esses valores, em termos de truncamento de números de ponto flutuante, pode nem haver redução no valor médio e sim nos valores individualizados por tipo de transação. A versão utilizada foi a SQL Server 2022 (RTM) - 16.0.1000.6.

Worker(s)	TPS	Tempo de Resposta (Avg. s)	Tempo de Transação (Avg. s)	Total Execution	Total Rows	Total Erros
1	174,231	0,001	0,006	20880	1405268	0
2	390,550	0,001	0,005	46588	3155653	0
4	588,212	0,001	0,006	70084	4761558	0

Assim como foi mencionado em parágrafos anteriores, os resultados podem estar indicando que o conjunto, hardware mais software, poderiam ter sido ainda mais explorados dado que os números ainda estavam crescendo. Na tabela acima, as observações são praticamente as mesmas feitas para o PostgreSQL, evolução do número de workers coincide com a evolução dos valores de TPS. Contudo, há de se destacar o desempenho

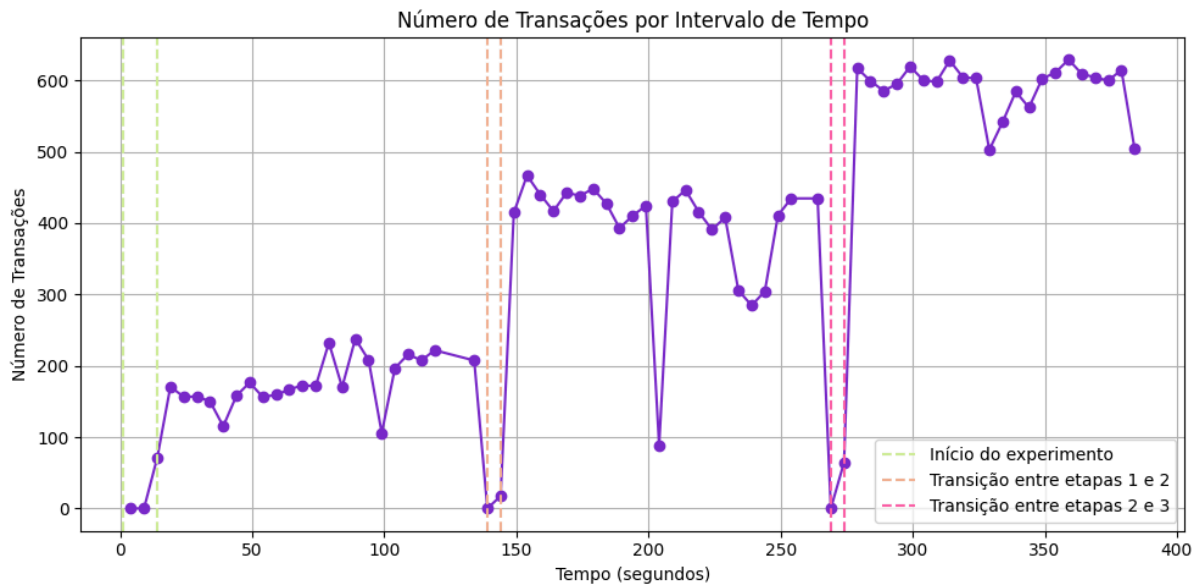
do SQL Server tendo evoluído 124,15% entre 1 e 2 workers, contra 83,25% do PostgreSQL. Ao olhar para a evolução entre 1 e 4 workers, o SQL Server evoluiu 237,6% enquanto o PostgreSQL evoluiu 204,64%. O tempo médio de resposta do SQL Server se mostrou muito estável e superior ao do PostgreSQL, provavelmente pelo fato dos testes terem sido realizados em um ambiente Windows que pertence a mesma corporação.

Na tabela abaixo são exibidos os valores por cada tipo de transação. Como os valores totais de transações no SQL Server foi maior, os valores por transação nessa tabela vão ser consideravelmente superiores. Contudo, há de se observar que a ferramenta Benchmark Factory mantém as proporções para cada tipo de transação, o que garante confiabilidade às comparações.

Transaction Mix		
Transaction Name	Transaction Count	Mix %
Trade-Update	1391	1,985
Trade-Status	13188	18,817
Trade-Result	6941	9,904
Trade-Order	7637	10,897
Trade-Lookup	5567	7,943
Security-Detail	9718	13,866
Market-Watch	12514	17,856
Market-Feed	691	0,986
Customer-Position	9027	12,880
Broker-Volume	3410	4,866

O gráfico abaixo mostra os valores de transações por recorte de tempo nas etapas de execução para o SQL Server. Algo de se observar, inicialmente, é que o SQL Server realiza um início de operações de forma suavizada. Nas três etapas é possível perceber um primeiro número de transações relativamente baixo e logo evolui para uma faixa mais estável e bem mais alta. Como o número de transações foi maior neste SGBD, as subidas são sempre mais íngremes e soam mais impactantes que no PostgreSQL, talvez por isso a observação anterior tenha ocorrido. No instante de tempo 204 do experimento, houve uma queda brusca no TPS para 88,4 seguido de um tempo máximo de resposta que saiu de 15 (no instante de tempo 204) para 4528 segundos (no instante de tempo 209). Esse tempo alto pode ter sido um ou mais transações que demoraram a finalizar e, conseqüentemente, outras ficaram aguardando para serem executadas. Dado que o Benchmark Factory é

obscuro nas formas como configura o banco, e seu log de execução é demasiado simples, não é possível afirmar nada com certeza sobre as causa dessa queda tão brusca.



Experimentos Após a Criação de Novos Índices

Os experimentos estiveram o tempo todo sobre nossa regência, mas a execução ficou por conta do Benchmark Factory. Dessa forma, as otimizações a se realizarem nos bancos, ficam limitadas ou carecem de alguma solução improvisada. Os benchmark realizados na ferramenta tem duas grandes fases importantes: Gerenciamento do banco e Procedimentos experimentais (A ferramenta utiliza outra nomenclatura cuja qual não lembro neste momento). Na parte de gerenciamento do banco, é possível definir tipo de banco, conexão, etc. e se haverá criação do banco, criação de backup, remoção do banco ao finalizar o experimento, etc. Ainda nessa fase, uma solução improvisada é criar um banco de dados populado e pular a parte de execução do benchmark, de forma que o banco não será alterado. Dessa forma, teremos um banco para fazer alterações com uma ferramenta externa como o DBeaver, por exemplo. Contudo, é importante fazer o máximo de alterações que deseje antes de realizar qualquer teste, pois o TPC-E altera o banco ao ser executado, então, a cada ajuste fino que se faça no banco e, conseqüentemente, o benchmark, um novo banco populado precisará ser criado.

Por conta de limite de tempo, dado estarmos utilizando um computador de um terceiro e precisaríamos devolver naquele mesmo dia, optamos por realizar a criação de índices em tabelas afetadas tanto por SELECTs quanto por UPDATEs. Este último pois, para realizar uma atualização, algum frame anterior realizou uma leitura em busca dos dados a serem alterados.

Como a estrutura dos bancos e os dados são os mesmo, os processos gerados pelo Benchmark Factory são os mesmos, pensamos em criar índices iguais nos dois bancos de dados. Outro detalhe importante é que os índices foram construídos na forma mais trivial possível, para evitar que parâmetros usados exclusivamente por um SGBD não impactasse nos resultados em comparação com o outro.

Por exemplo, com base na transação “Trade-Order”, foram pensados algumas ideias de criação de índices da seguinte forma:

```
CREATE INDEX IX_CustomerAccount_CA_ID  
ON dbo.E_CUSTOMER_ACCOUNT (CA_ID);
```

Foca em otimizar a consulta que busca informações na tabela CUSTOMER_ACCOUNT usando a coluna CA_ID. É uma coluna frequentemente utilizada em muitas das transações para operações de busca, e como é uma chave primária, um índice seria interessante para melhorar a performance.

```
CREATE INDEX IX_Customer_C_ID ON dbo.E_CUSTOMER (C_ID);
```

A tabela CUSTOMER é acessada usando a coluna C_ID para obter informações do cliente. Este índice tem como objetivo acelerar a busca de clientes com base no ID.

```
CREATE INDEX IX_Broker_B_ID ON dbo.E_BROKER (B_ID);
```

A coluna B_ID na tabela BROKER é usada para buscar informações sobre o corretor. Um índice nesta coluna irá melhorar a performance das consultas que envolvem a busca de brokers.

```
CREATE INDEX IX_AccountPermission_AP_CA_ID_FullName  
ON dbo.E_ACCOUNT_PERMISSION (AP_CA_ID, AP_F_NAME, AP_L_NAME,  
AP_TAX_ID);
```

A ideia deste índice, é cobrir a consulta que verifica permissões, utilizando AP_CA_ID, AP_F_NAME, AP_L_NAME e AP_TAX_ID. Como várias colunas são usadas na cláusula WHERE, um índice composto seria uma tentativa interessante de acelerar esses tipos de consultas.

```
CREATE INDEX IX_HoldingSummary_HS_CA_ID_S_SYMB  
ON dbo.E_HOLDING_SUMMARY (HS_CA_ID, HS_S_SYMB);
```

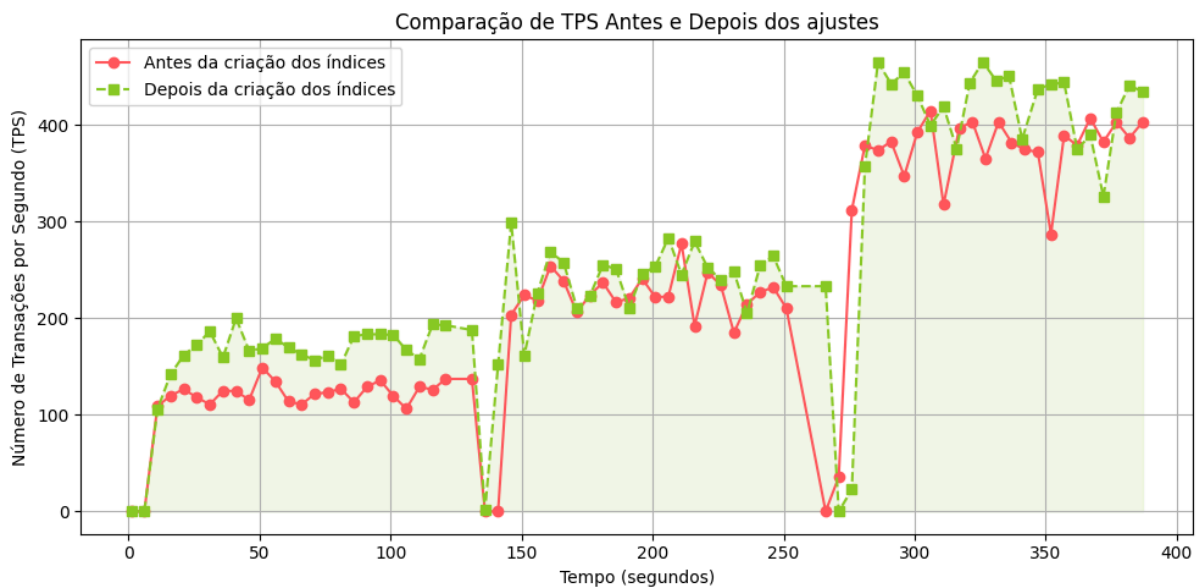
Esse índice é útil para as consultas que buscam a quantidade de ações em HOLDING_SUMMARY com base em HS_CA_ID e HS_S_SYMB. Ele melhora a performance ao lidar com a busca de holdings para um cliente específico por símbolo.

```
CREATE INDEX IX_Holding_H_CA_ID_S_SYMB  
ON dbo.E_HOLDING (H_CA_ID, H_S_SYMB, H_DTS);
```

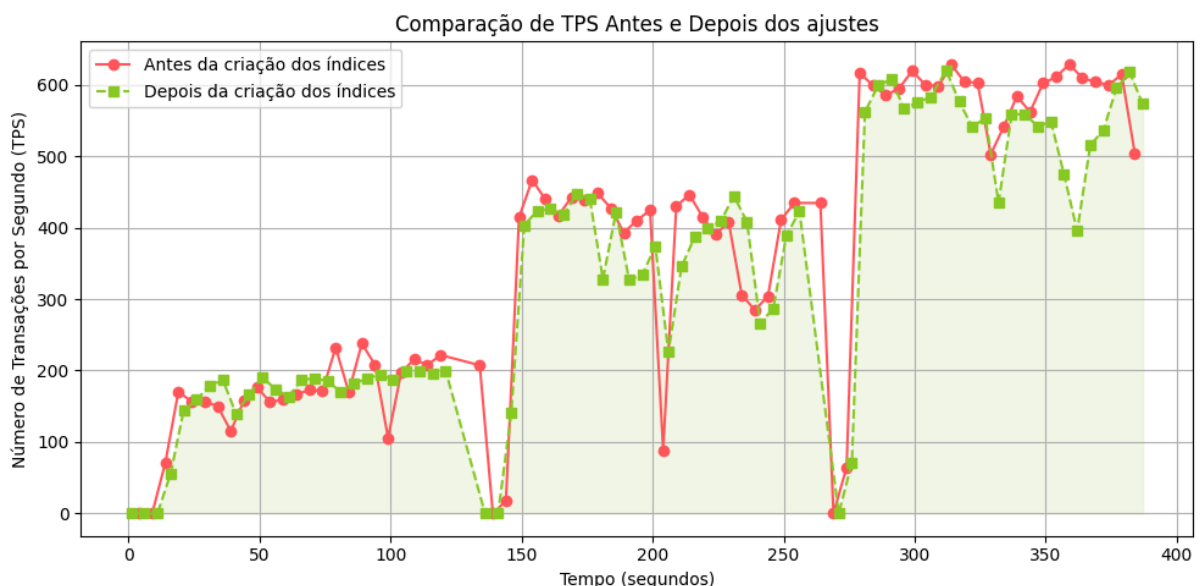
O índice tentará cobrir as consultas que acessam a tabela HOLDING utilizando H_CA_ID, H_S_SYMB e ordenando por H_DTS. Isso ajuda na performance durante a recuperação de dados de holdings, especialmente considerando as operações de venda e compra.

Para todos os outros tipos de transações, foram pensados índices com as mesmas análises técnicas e focando no suporte às leituras que são necessárias, tanto para a realização de UPDATES quanto para os próprios SELECTs. Abaixo, no Gráfico, podemos ver os resultados para o número de transações por segundo do PostgreSQL. A área verde,

realça a evolução dos resultados a partir dos índices criados. O destaque fica na execução com apenas um worker, na qual todos os pontos após a criação dos índices apontaram para uma melhoria no número de transações por segundo.



Como mencionado anteriormente, foram feitos os mesmos ajustes para o SQL Server por questões de tempo e de limitações do Benchmark Factory. Contudo, os sinais de melhoria foram mais fortemente percebidos no PostgreSQL pois, no SQL Server, em alguns pontos até piorou. Na imagem abaixo, mostramos os resultados fazendo a mesma comparação anterior entre o experimento antes da criação dos índices contra os experimentos após a criação dos índices.



Contudo, é crucial destacar que esses valores não possuem peso estatístico dado o fato de que foi realizada uma única execução por número de workers para cada estado do banco, original e com índices criados. Para que se possa validar ou invalidar a criação dos

índices nesse caso, seria necessária uma bateria de experimentos como, por exemplo, 30 execução com cada número de workers. A partir daí, seria possível garantir que os resultados não iriam variar mais do que o assistido nessa bateria de experimentos.

Contudo, ao analisar as transações individualmente, podemos extrair alguns dados interessantes. Na Tabela abaixo, fazemos a comparação entre o banco do PostgreSQL original, ou seja, assim que é gerado e populado, e após a criação dos índices visando melhora na performance. As métricas que se encontram na tabela estão dispostas para cada tipo de transação. Dessa forma, podemos perceber que, para alguns tipos de transações, mesmo o tempo anterior sendo consideravelmente ótimo, há uma melhora nas médias e nos valores totais. Por exemplo, no caso da transação “Trade-Lookup”, uma transação que faz muitas leituras, a inserção dos índices nas tabelas impactadas resultou em uma melhora de 92,32% em relação ao resultado anterior. Essa melhora também pode ser assistida para a transação “Broker-Volume” que também é exclusivamente de leitura. No entanto, analisando os valores para a transação “Trade-Order”, podemos perceber que não resultou em melhora a criação de índices, possivelmente, devido ao fato da transação ter um peso considerado na inserção ao invés de leitura.

	Original			Alterado		
Transaction Name	Min	Avg	Max	Min	Avg	Max
Trade-Update	0,001	0,001	0,069	0,001	0,002	0,050
Trade-Status	0,001	0,001	0,027	0,001	0,001	0,054
Trade-Result	0,001	0,001	0,042	0,001	0,001	0,012
Trade-Order	0,001	0,001	0,003	0,001	0,001	0,010
Trade-Lookup	0,001	0,001	0,717	0,001	0,001	0,055
Security-Detail	0,001	0,001	0,010	0,001	0,001	0,012
Market-Watch	0,001	0,035	0,127	0,001	0,029	0,165
Market-Feed	0,001	0,001	0,002	0,001	0,001	0,005
Customer-Position	0,001	0,001	0,012	0,001	0,001	0,010
Broker-Volume	0,001	0,004	0,022	0,001	0,003	0,017

Também fizemos essa mesma comparação para os resultados da otimização do SQL Server. Na Tabela abaixo apresentamos esse resultados, cujos quais refletem dados parecidos com os observados no PostgreSQL, possivelmente pelo fato dessas métricas já serem bastante interessantes mesmo antes da criação dos índices. Porém, há de se destacar um fato importante nesses resultados, a criação dos índices no SQL Server não impactaram em nada na média tempo nem no mínimo do tempo de resposta. Os detalhes ficam por conta dos valores máximos de tempo de resposta, que, no caso da transação

“Customer-Position”, apresenta uma melhora de 96,27% e 53,06% para a transação “Trade-Lookup”. Interessante também apresentar o caso da transação “Trade-Order” que, no PostgreSQL, teve uma piora com a criação dos índices, no SQL Server houve uma melhora de 47,61%.

	Original			Alterado		
Transaction Name	Min	Avg	Max	Min	Avg	Max
Trade-Update	0,001	0,001	0,038	0,001	0,001	0,037
Trade-Status	0,001	0,001	0,019	0,001	0,001	0,082
Trade-Result	0,001	0,001	0,042	0,001	0,001	0,042
Trade-Order	0,001	0,001	0,021	0,001	0,001	0,011
Trade-Lookup	0,001	0,001	0,098	0,001	0,001	0,046
Security-Detail	0,001	0,001	0,039	0,001	0,001	0,036
Market-Watch	0,001	0,001	0,013	0,001	0,001	0,021
Market-Feed	0,001	0,001	0,009	0,001	0,001	0,011
Customer-Position	0,001	0,001	0,589	0,001	0,001	0,022
Broker-Volume	0,001	0,001	0,105	0,001	0,001	0,083

CONCLUSÃO

Este trabalho apresentou experimentos no benchmark TPC-E, um benchmark criado para simular cargas de trabalho de processamento de transações OLTP, com foco em operações financeiras.

Inicialmente, pretendíamos fazer um código para realizar as transações e, dessa forma, ter mais controle sobre as métricas que coletaríamos. Contudo, limitações técnicas e de tempo nos levaram a utilizar a ferramenta Benchmark Factory. O uso cuidadoso da ferramenta nos fez observar pontos fortes e pontos fracos da mesma, mas para este trabalho ela foi fundamental.

Os resultados mostram que a implementação de índices nos dois bancos, resultou em melhorias pontuais, mas um ajuste fino de forma global necessitaria um estudo muito aprofundado de cada uma das transações que são executadas, da forma como o banco foi construído, etc. Também pudemos perceber que o SQL Server se destacou em termos numéricos, possivelmente pelo fato de estar rodando em um ambiente Windows e com sistema de arquivos NTFS.

Pudemos perceber também que, para que os dados deste trabalho tenham poder estatístico, o número de execuções deveria ser maior e promover uma comparação em dois ou mais Sistemas Operacionais para não favorecer nenhum dos SGBDs.

REFERÊNCIAS

- [1] DO NASCIMENTO, Rilson Oscar. **DBT-5: Uma Implementação de Código Aberto do TPC-E para Avaliação de Desempenho de Sistemas de Processamento Transacional**. 2008. Tese de Doutorado. Dissertação de mestrado, Centro de Informática da Universidade Federal de Pernambuco, UFPE.
- [2] TPC www.tpc.org. TPC Benchmark(tm) C Standard Specification, August 1992.
- [3] TPC www.tpc.org. TPC Benchmark(tm) E Standard Specification, August 2007.
- [4] GUARDA, Simone Dutra Martins. Desenvolvimento de um método para integrar um segmentador de grandes imagens no banco de dados PostgreSQL. 2019.
- [5] *HammerDB*, www.hammerdb.com/. Accessed 2 Aug. 2025.
- [6] docs.aws.amazon.com/aurora-dsdl/latest/userguide/working-with.html. Accessed 2 Aug. 2025.
- [7] DOURADO, George Gabriel Mendes. **Contribuindo para a Avaliação do Teste de Programas Concorrentes: uma abordagem usando benchmarks**. 2015. Tese de Doutorado. Universidade de São Paulo.
- [8] “NTFS.” *Wikipedia*, Wikimedia Foundation, 15 June 2024, pt.wikipedia.org/wiki/NTFS.